

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

**Отчет по лабораторной работе №3
по дисциплине «Системы искусственного интеллекта»**

Выполнил:
студент группы ИСТбд-31 Коняхин Н.С

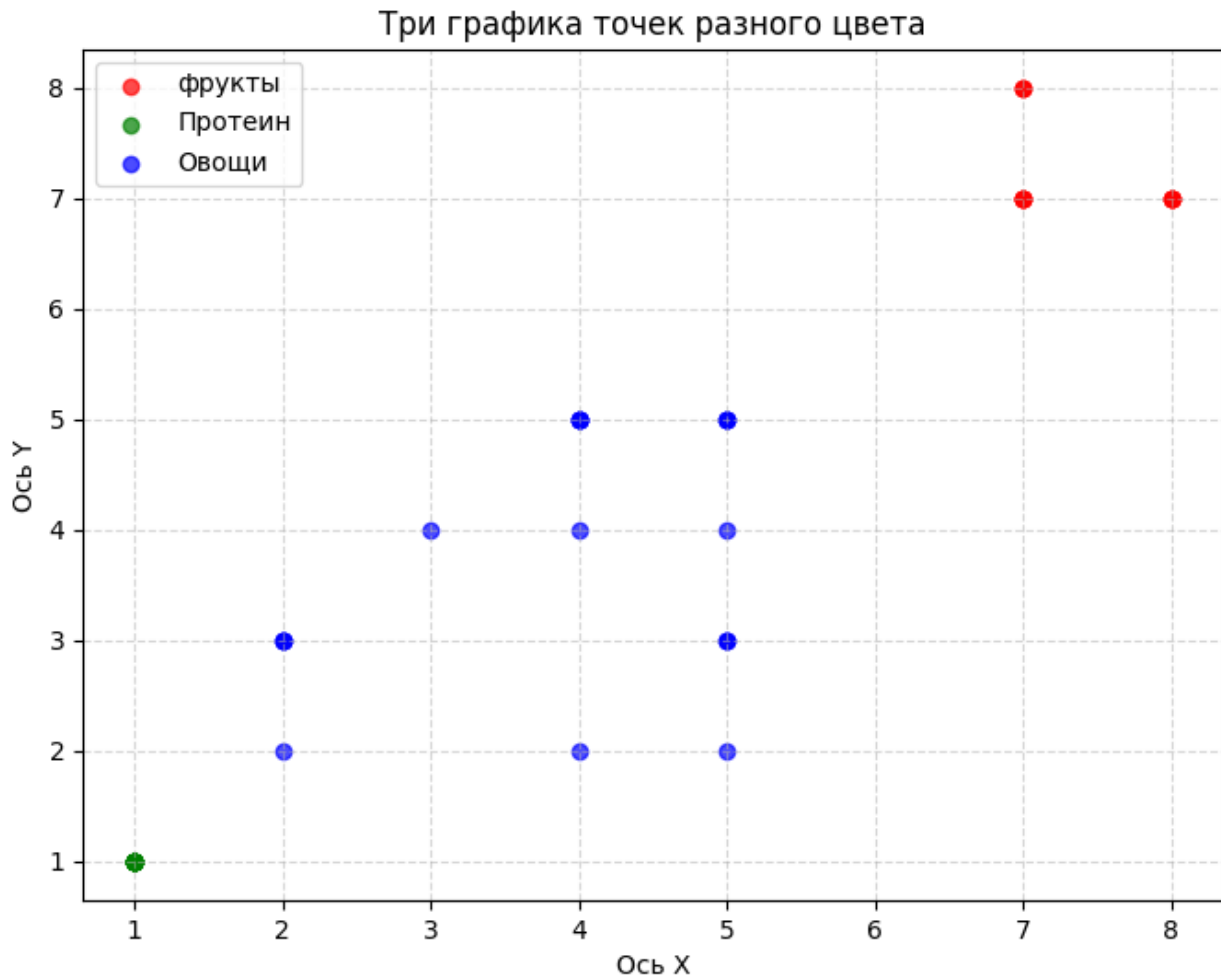
Проверил:
Преподаватель Шишкин В.В.

Ульяновск, 2026

Цель:

Разработать алгоритм KNN в самописном варианте и сравнить с работой модели из библиотеки SCikit-learn. Рассмотреть работу на разном наборе данных

1. Набор данных



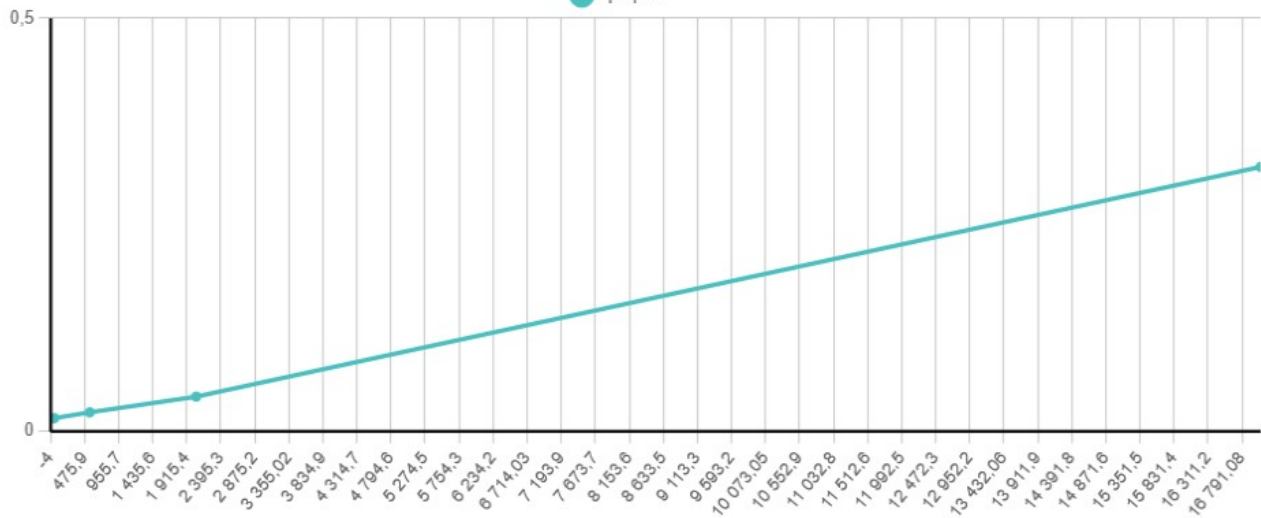
результат работы нейросети:

```
Результаты классификации:  
Метрики: [7 8] Истинна: фрукт -- фрукт :Модель  
Метрики: [8 7] Истинна: фрукт -- фрукт :Модель  
Метрики: [7 7] Истинна: фрукт -- фрукт :Модель  
Метрики: [1 1] Истинна: протеин -- протеин :Модель  
Метрики: [1 1] Истинна: протеин -- протеин :Модель  
Метрики: [5 5] Истинна: овощ -- овощ :Модель  
Метрики: [4 4] Истинна: овощ -- овощ :Модель  
Метрики: [2 2] Истинна: овощ -- овощ :Модель  
Метрики: [7 7] Истинна: фрукт -- фрукт :Модель  
Метрики: [1 1] Истинна: протеин -- протеин :Модель  
Точность KNN: 100.00%
```

результат работы самописного алгоритма:

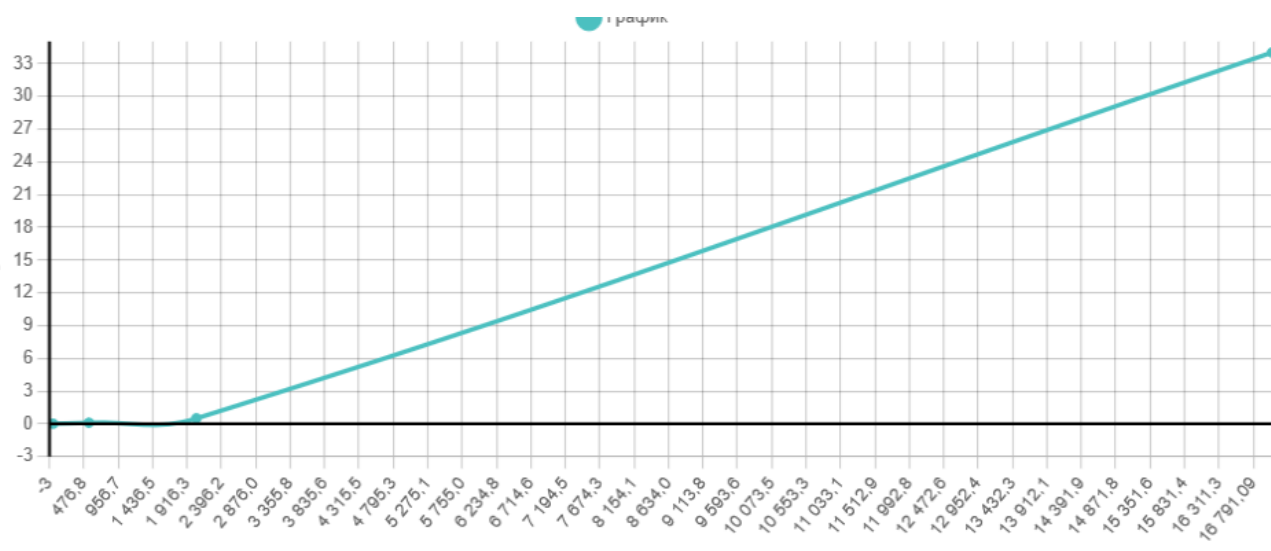
```
метрики: (8, 7) Результат: фрукт -- Соседи: ['фрукт', 'фрукт', 'фрукт', 'фрукт', 'фрукт']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (3, 5) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (4, 2) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (3, 3) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (5, 4) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (4, 4) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
```

графики работы нейросети



№	x	y	Действия
1	50	0.016	▲ ▼ Удалить
2	550	0.023	▲ ▼ Удалить
3	2050	0.042	▲ ▼ Удалить
4	17050	0.32	▲ ▼ Удалить

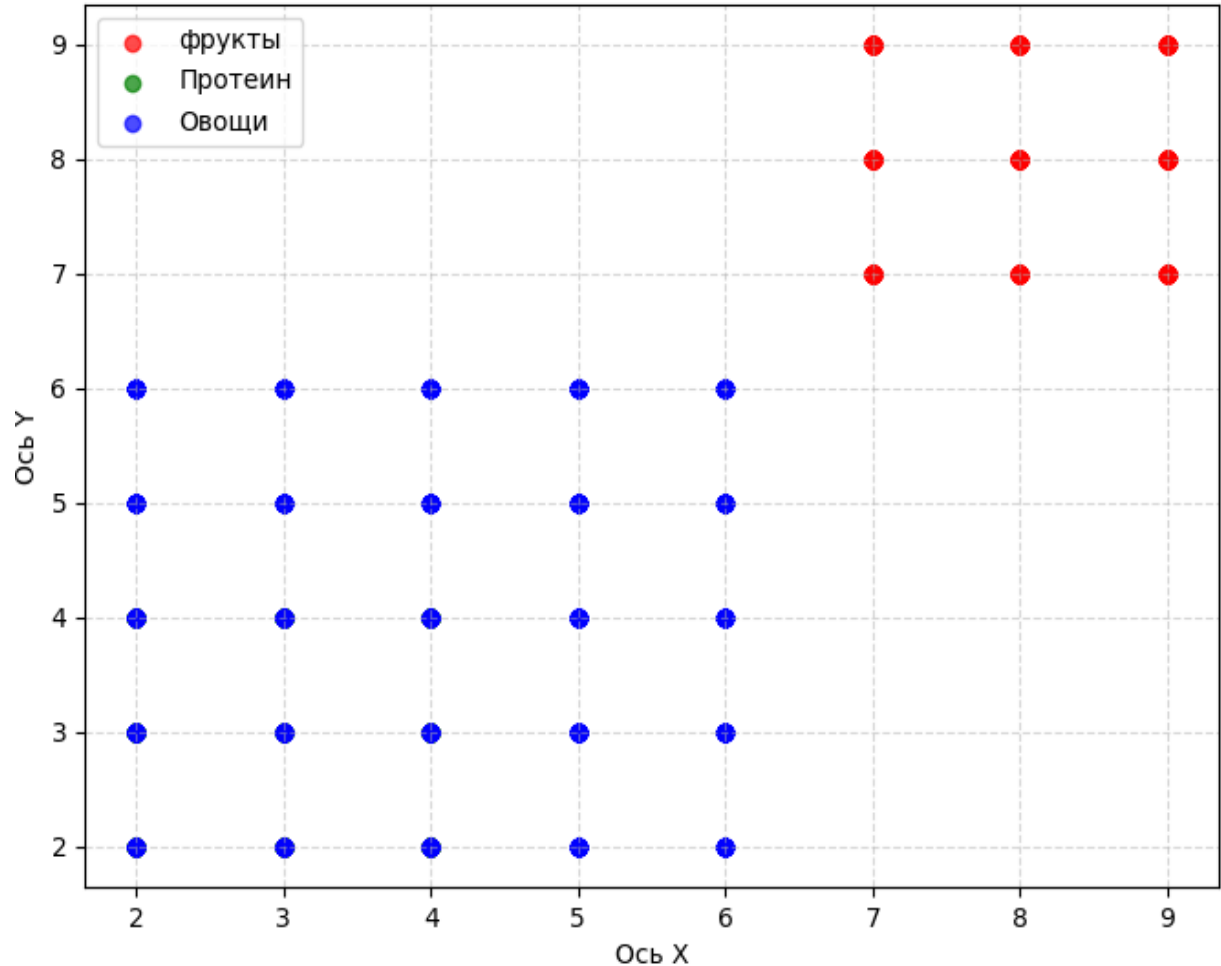
График работы алгоритма



№	x	y	Действия
1	50	0.0029	▲ ▼ Удалить
2	550	0.1	▲ ▼ Удалить
3	2050	0.49	▲ ▼ Удалить
4	17050	34	▲ ▼ Удалить

2.Набор данных

Три графика точек разного цвета



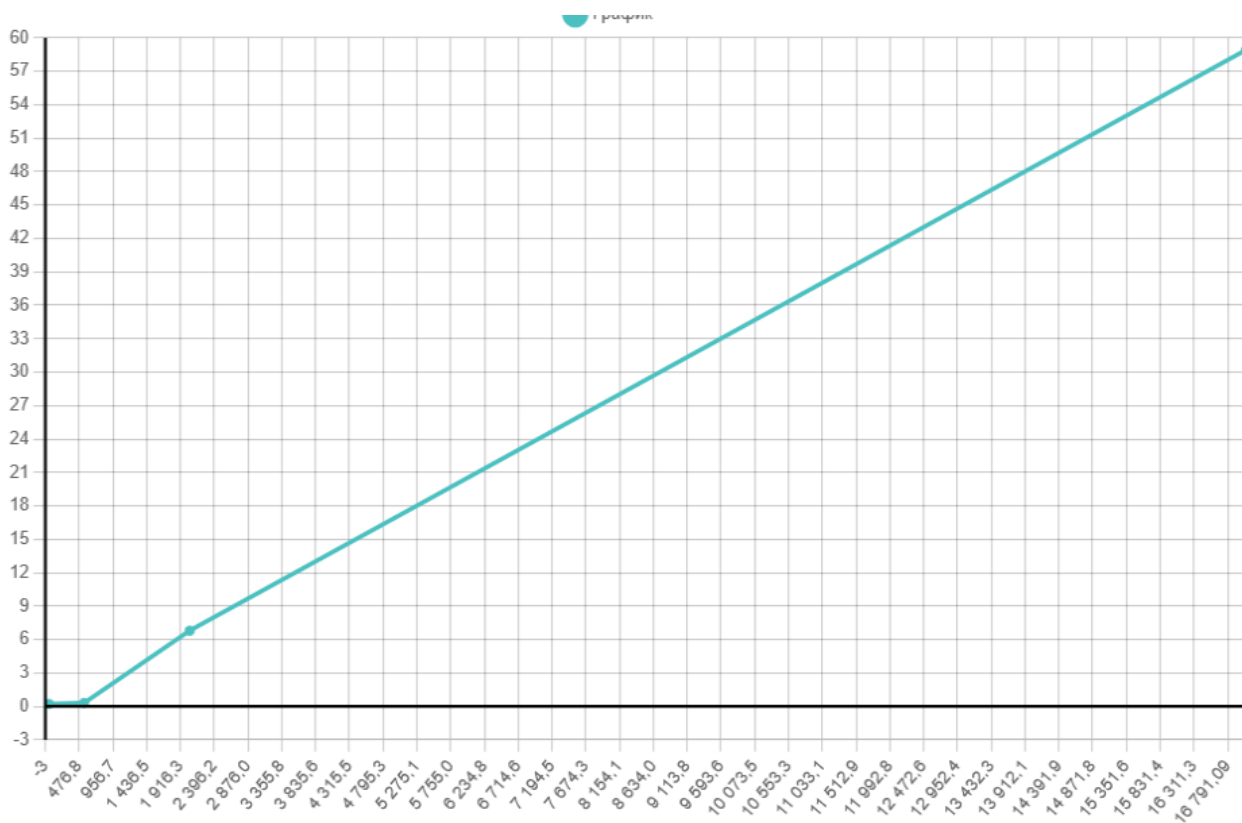
Работа нейросети

```
Метрики: [2 3] Истинна: овощ -- протеин :Модель
Метрики: [5 6] Истинна: овощ -- овощ :Модель
Метрики: [8 8] Истинна: фрукт -- фрукт :Модель
Метрики: [5 4] Истинна: овощ -- овощ :Модель
Метрики: [3 3] Истинна: овощ -- овощ :Модель
Метрики: [3 4] Истинна: овощ -- протеин :Модель
Метрики: [4 2] Истинна: овощ -- протеин :Модель
Метрики: [8 7] Истинна: фрукт -- фрукт :Модель
Метрики: [8 8] Истинна: фрукт -- фрукт :Модель
Метрики: [3 4] Истинна: протеин -- протеин :Модель
Точность KNN: 81.38%
0.2707028388977051
```

Работа Алгоритма

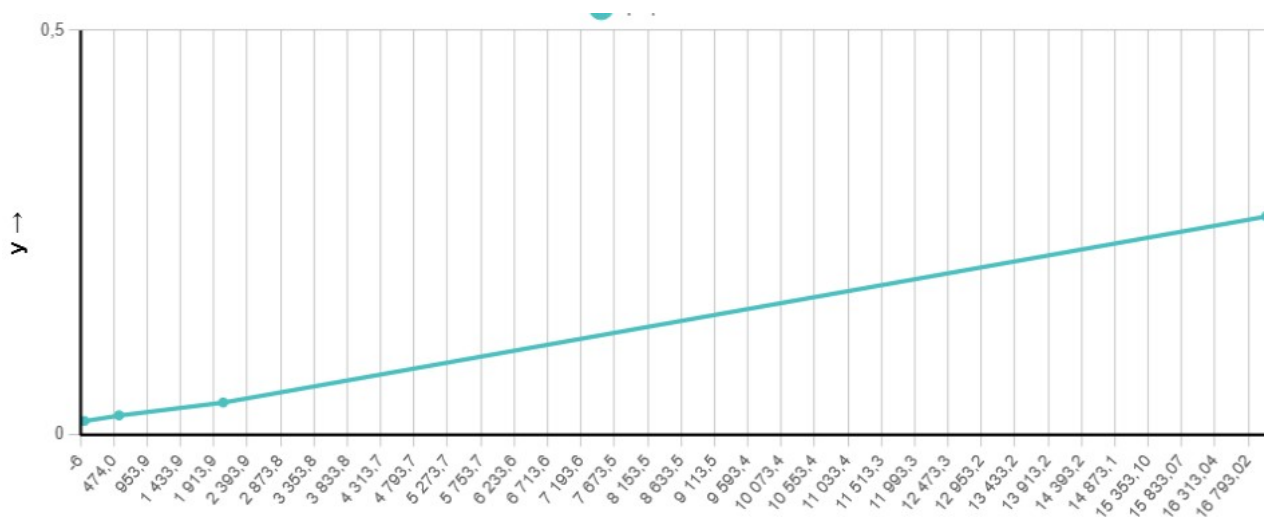
```
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
метрики: (8, 7) Результат: фрукт -- Соседи: ['фрукт', 'фрукт', 'фрукт', 'фрукт', 'фрукт']
метрики: (2, 2) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (2, 5) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (2, 4) Результат: овощ -- Соседи: ['овощ', 'овощ', 'овощ', 'овощ', 'овощ']
метрики: (1, 1) Результат: протеин -- Соседи: ['протеин', 'протеин', 'протеин', 'протеин', 'протеин']
```

График работы алгоритма



№	x	y	Действия
1	50	0.2	▲ ▼ Удалить
2	550	0.29	▲ ▼ Удалить
3	2050	6.79	▲ ▼ Удалить
4	17050	58.9	▲ ▼ Удалить

график работы нейросети



№	x	y	Действия
5	50	0.017	▲ ▼ Удалить
6	550	0.024	▲ ▼ Удалить
7	2050	0.04	▲ ▼ Удалить
8	17050	0.27	▲ ▼ Удалить

3 Набор данных

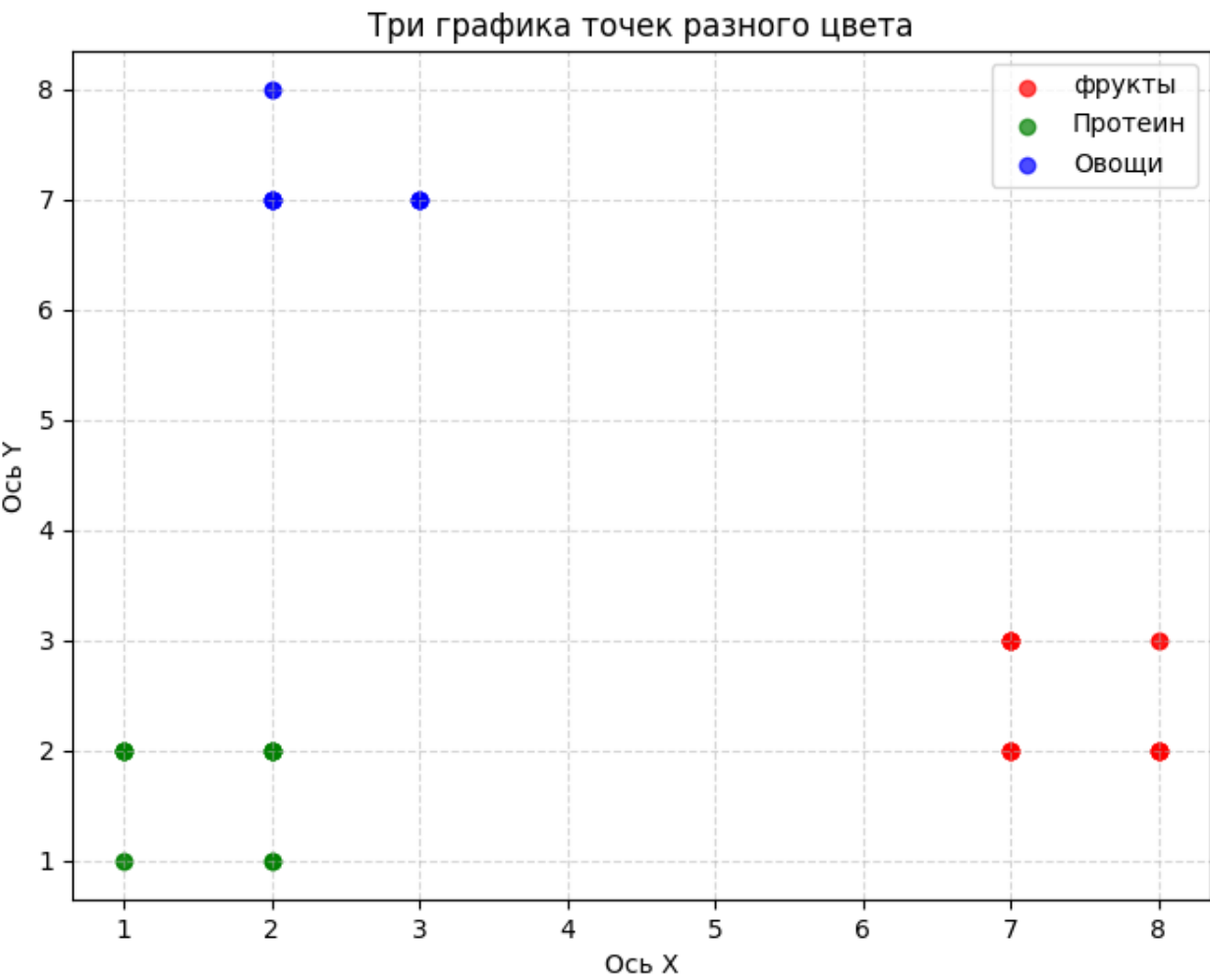
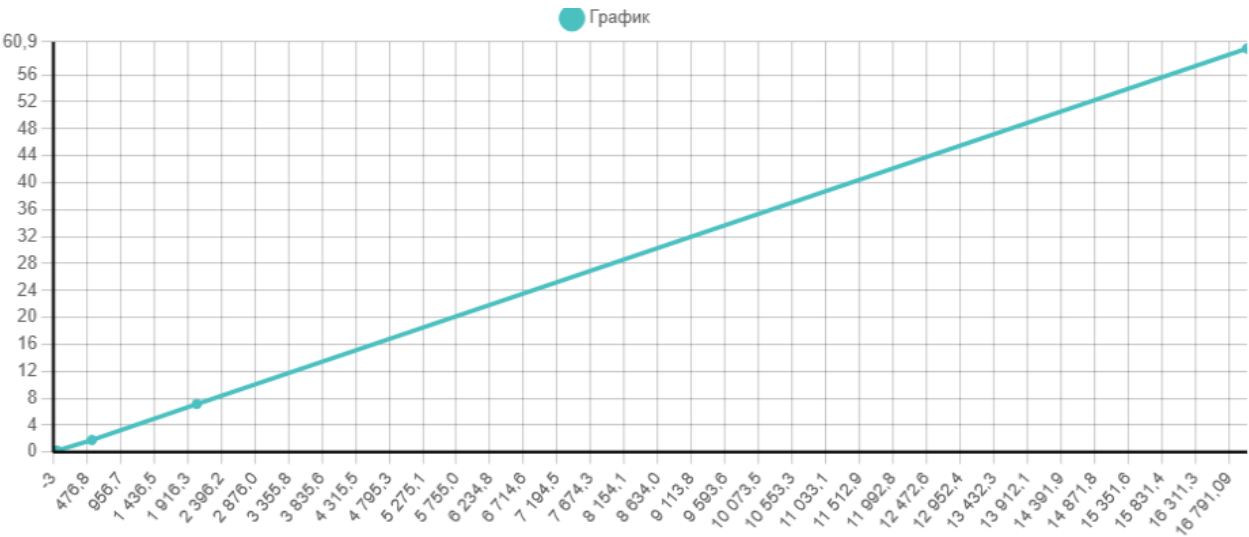
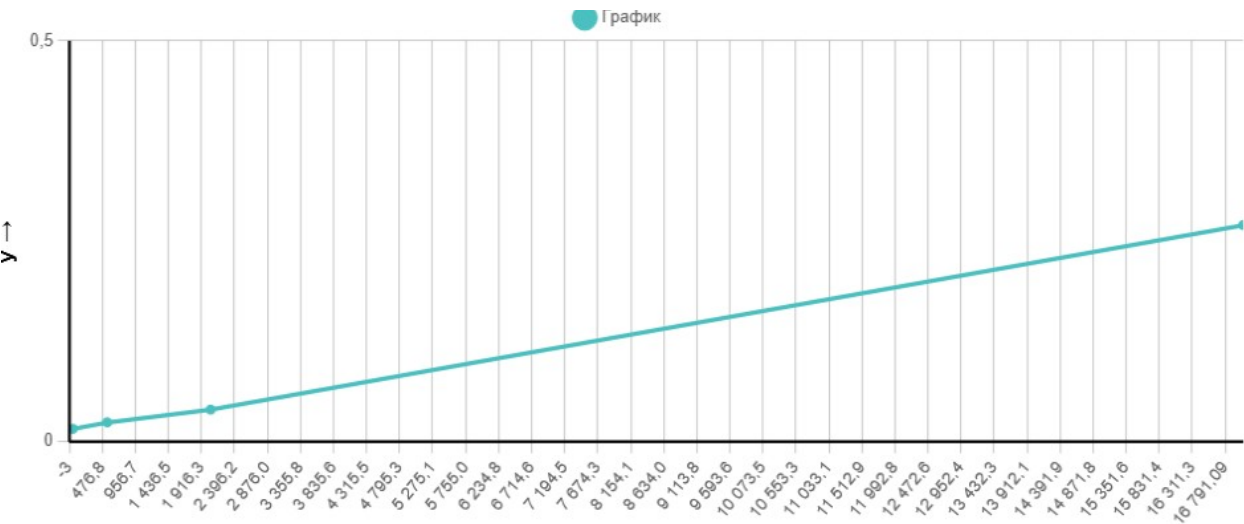


График алгоритма по времени



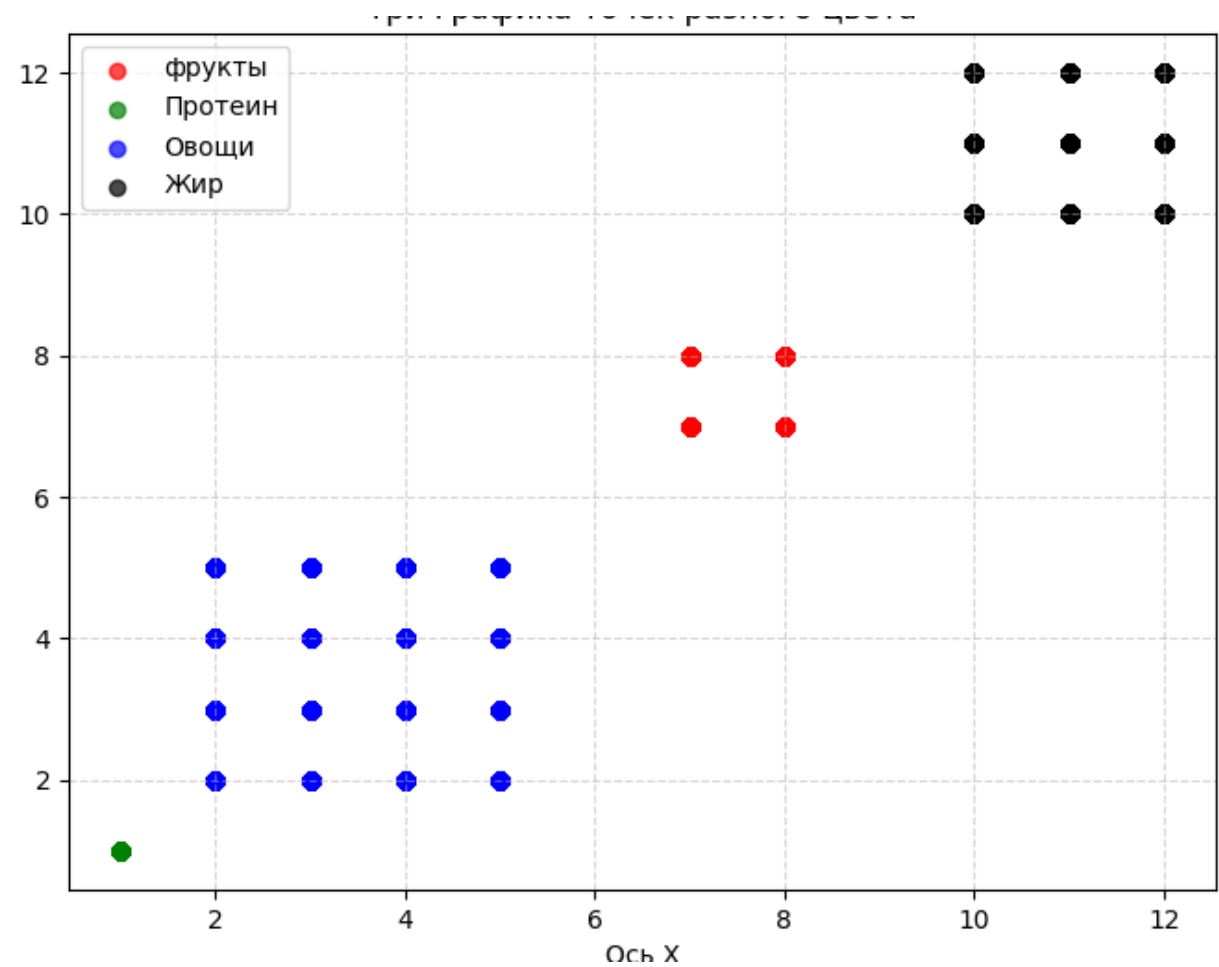
№	x	y	Действия
1	50	0.2	▲ ▼ Удалить
2	550	1.77	▲ ▼ Удалить
3	2050	7.11	▲ ▼ Удалить
4	17050	59.9	▲ ▼ Удалить

График нейросети по времени

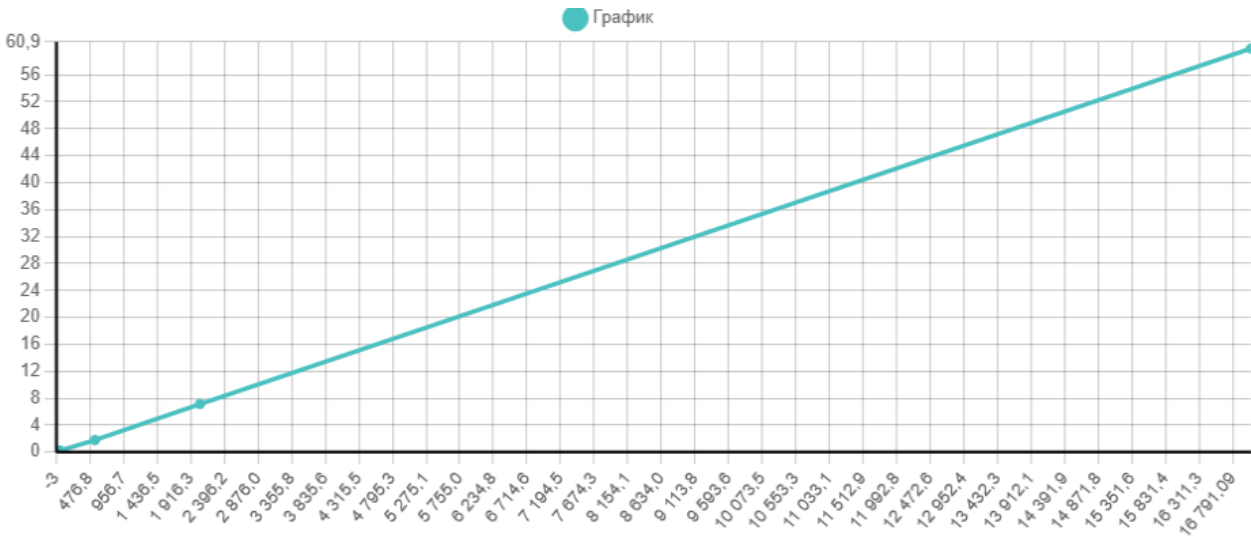


№	x	y	Действия
5	50	0.016	▲ ▼ Удалить
6	550	0.024	▲ ▼ Удалить
7	2050	0.04	▲ ▼ Удалить
8	17050	0.27	▲ ▼ Удалить

Наборы данных с доп классом

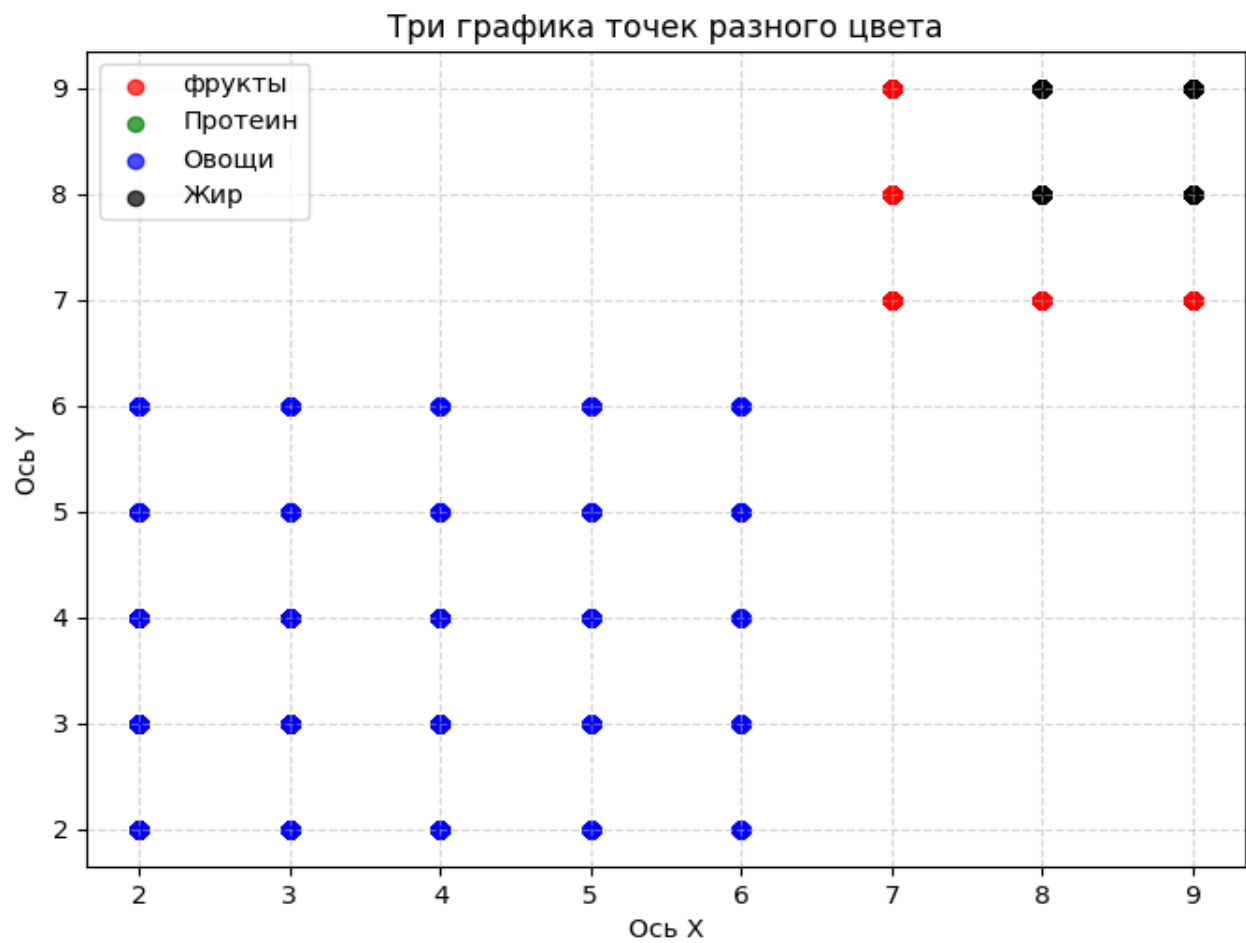
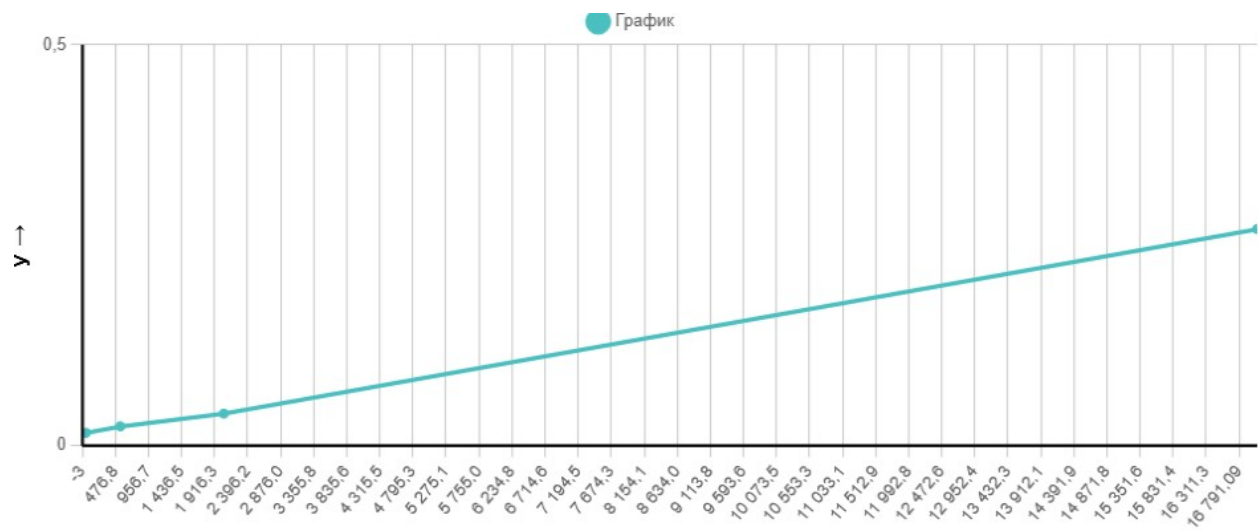


работа алгоритма

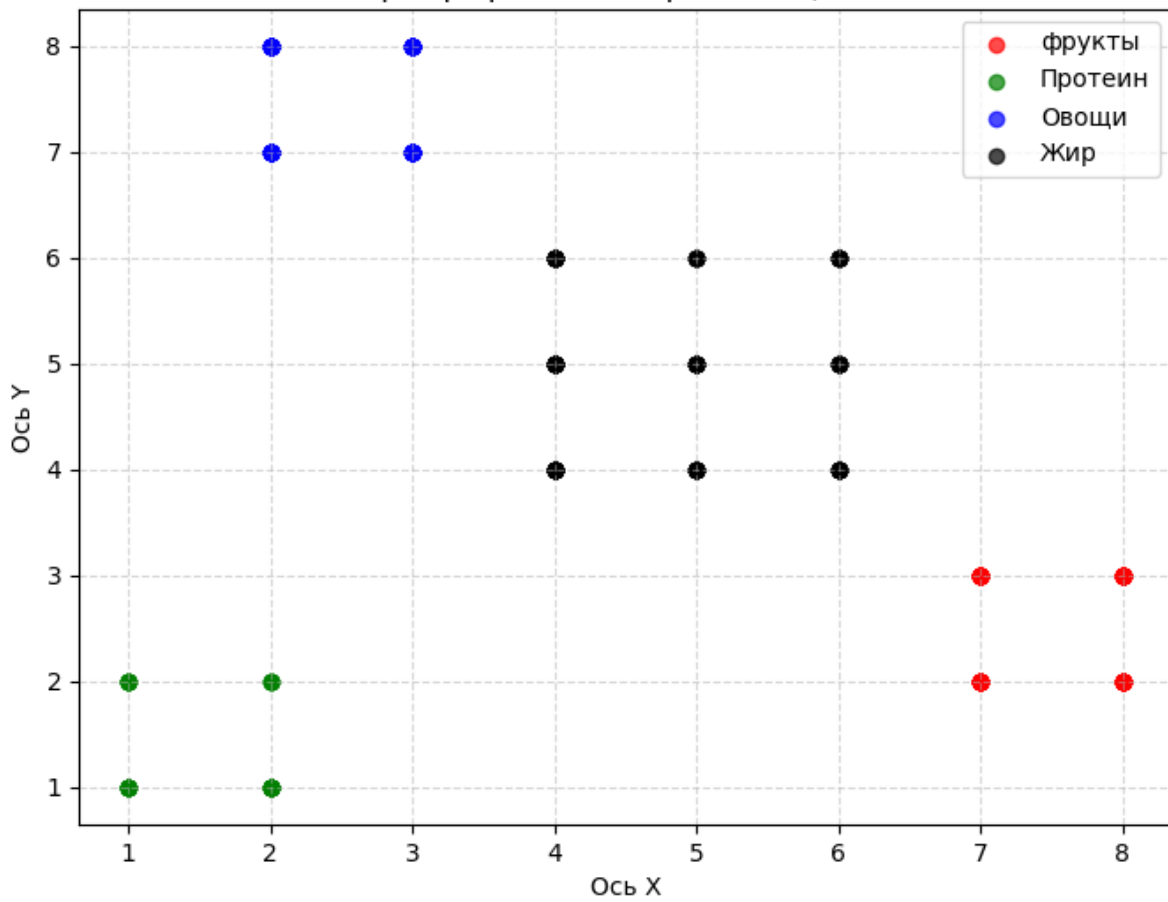


Работа алгоритма на данном наборе данных оказалось самым большим

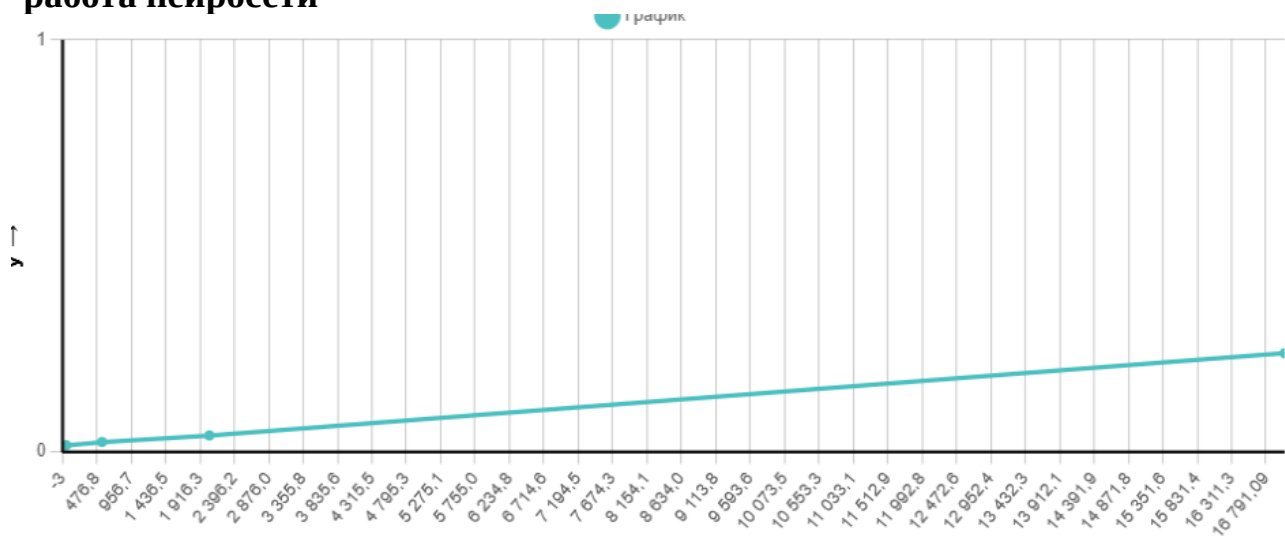
Работа нейросети



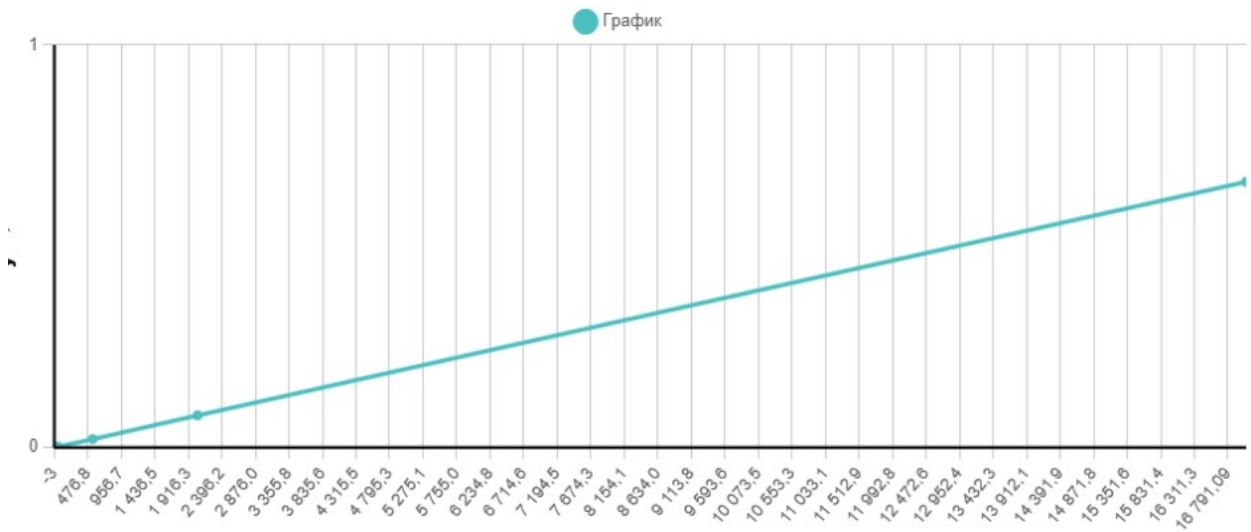
Три графика точек разного цвета



работа нейросети



Работа алгоритма



время выполнения оказалась одинаковой для 3 наборов данных однако для второго варианта точность модели снижалась так как данные наложены друг на друга.

Вывод :

Работа самописного алгоритма заметно хуже даже на небольших размерах данных по сравнению с библиотекой Scikit . Модель из библиотеки на разных наборах данных показывает себя с наилучшей стороны. Повторный эксперимент с дополнительным классом показал что модель остается лидером .

```
import random as r
import time

import matplotlib.pyplot as plt
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

line = [","]

fruit_x = []
fruit_y = []

protein_x = []
protein_y = []

veg_x = []
veg_y = []
```

```

fat_x = []
fat_y = []
FILE = 'eat.csv'
SIZE = 40

with open(FILE, 'r', encoding='UTF-8') as file:
    while len(line) != 1:
        try:
            line = file.readline().split(',')
            if line[2][:-1] == 'овощ':
                veg_x.append(int(line[0]))
                veg_y.append(int(line[1]))

            elif line[2][:-1] == 'фрукт':
                fruit_x.append(int(line[0]))
                fruit_y.append(int(line[1]))
            elif line[2][:-1] == 'протеин':
                protein_x.append(int(line[0]))
                protein_y.append(int(line[1]))
            elif line[2][:-1] == 'жир':
                fat_x.append(int(line[0]))
                fat_y.append(int(line[1]))
        except:
            continue

plt.figure(figsize=(8, 6))

# Строим scatter-графики с разными цветами и метками
plt.scatter(fruit_x, fruit_y, color='red', label='фрукты', alpha=0.7)
plt.scatter(protein_x, protein_y, color='green', label='Протеин', alpha=0.7)
plt.scatter(veg_x, veg_y, color='blue', label='Овощи', alpha=0.7)
plt.scatter(fat_x, fat_y, color='black', label='Жир', alpha=0.7)

# Добавляем заголовок и подписи осей
plt.title('Три графика точек разного цвета')
plt.xlabel('Ось X')
plt.ylabel('Ось Y')

# Показываем легенду
plt.legend()

# Отображаем сетку для удобства
plt.grid(True, linestyle='--', alpha=0.5)

def scikit_knn(file):
    # 1. Создаём данные (как в прошлом примере)
    df = pd.read_csv(file)

    print(df.head( n = 5)) # n кол-во строк в консоль

    info = df.drop('класс', axis=1)
    label = df['класс']

    info = info.reset_index(drop=True)
    label = label.reset_index(drop=True)

    info_train, info_test, label_train, label_test = train_test_split(info, label, test_size = 0.2)
    # 2. Создаём модель KNN

```

```

# n_neighbors=5 -> значит, смотрим на 5 ближайших соседей
model = KNeighborsClassifier(n_neighbors=5 ,p=1)

# 3. "Обучаем" модель
# Для KNN это просто сохранение данных в памяти
model.fit(info_train, label_train)

# 4. Предсказываем
predictions = model.predict(info_test)
print("Результаты классификации:")
for right,answer,metr in zip(label_test,predictions,info_test.values):
    print(f'Метрики: {metr} Истинна: {right} -- {answer} :Модель' )

# 5. Проверяем точность
print(f"Точность KNN: {accuracy_score(label_test, predictions) * 100:.2f}%")

def manhattan_distance(x1,y1,x2,y2,label):
    return ( abs(x1-x2)+abs(y1-y2),label[:-1] )

def classificate(neibors_list):
    classes_dict = {}
    for name_class in neibors_list:

        if name_class not in classes_dict.keys():
            classes_dict[name_class] = 1
        else:
            classes_dict[name_class] += 1
    return max(classes_dict)

def selfmade_knn(file,dot_tup,k):
    # 1. загружаем точку
    # 2. находим расстояние для каждой другой точки(манхетенн)
    # 3. определяем ближайших K соседей
    # 4. опеределяем из сосодей кого больше (фрукт или протеин например) и выдаем
результат
    line = ["",""]
    distance_list = []
    k_neibors = []
    with open("eat.csv",'r',encoding='UTF-8') as file:

        while len(line) > 1:
            try:
                line = file.readline()
                line = line.split(',')

                file_sweet = int(line[0])
                file_crunch = int(line[1])
                label_eat = line[2]

            distance_list.append(manhattan_distance(x1=file_sweet,y1=file_crunch,x2=dot_tup[0],y2=dot
_tup[1],label=label_eat))

            except:
                continue
            distance_list.sort(key = lambda x : x[0])

            k_neibors = [ distance_list[i][1] for i in range(0,k)]

            return f'метрики: {dot tup} Результат: {classificate(k_neibors)} -- Соседи: { k_neibors

```



```
}'  
  
start_time = time.time()  
scikit_knn(FILE)  
end_time = time.time()  
res_time = end_time - start_time  
print(f'{res_time}')  
start_time = time.time()  
for i in range(0,SIZE):  
    class_eat = r.randint(1, 3)  
    if class_eat == 1:  
  
        sweet = r.randint(7, 8)  
        crunch = r.randint(7, 8)  
    elif class_eat == 2:  
  
        sweet = r.randint(2, 5)  
        crunch = r.randint(2, 5)  
    else:  
  
        sweet = r.randint(1, 1)  
        crunch = r.randint(1, 1)  
    eat_tup = (sweet,crunch)  
    print(selfmade_knn(FILE,eat_tup,k=5))  
end_time = time.time()  
res_time = end_time - start_time  
print(f'{res_time}')
```

